

CONCEPT PRIMER + VENDOR LANDSCAPE

Harness Engineering

The AI-Agent Sense: Scaffolding, Control Planes, and How the
DevOps/Platform Vendor Landscape Is Adopting It

Scope: What "harness engineering" means for LLM agents · Short profiles of 10 CI/CD, GitOps, and platform-
engineering vendors

Research date: July 2026

Part 1 — What "Harness Engineering" Means

Agent = Model + Harness. That equation is the entire concept. The model is a stateless token predictor — it produces text, nothing else. It cannot edit a file, run a command, remember yesterday's conversation, or know when to stop. The **harness** is everything else: the code that turns raw next-token prediction into an autonomous system that reliably takes action in the world. Anthropic is widely credited with popularizing the term ("agent harness" or "scaffolding") to describe this infrastructure layer, and by 2026 it has become standard vocabulary across every major lab shipping agents. **DOCUMENTED**

Why this matters more than model choice, in practice: in March 2026 the LangChain engineering team moved their coding agent from 30th to 5th place on Terminal-Bench 2.0 — without changing the underlying model at all. The entire gain came from optimizing the harness around it. Separately, GitHub's own published benchmarking of its Copilot agentic harness (June 2026) explicitly frames the harness — not the model — as the thing that determines token efficiency, speed, and predictability across Claude, GPT, and other models running through it. **DOCUMENTED**

The formal decomposition

2026 academic surveys have converged on a six-component formalization: **H = (E, T, C, S, L, V)** — Environment, Tools, Context, State, Loop/Logic, and Verification. Practitioners generally collapse this into a simpler five-layer mental model:

1. Tool orchestration — what actions the agent can take, and how calls are dispatched
2. Context & memory — what the model sees each turn; how state persists across long tasks
3. The execution loop — think → act → observe → think again, and when to stop
4. Guardrails — permissions, sandboxing, approval gates, scoped credentials
5. Observability — tracing, audit logs, cost controls, evals

Source: Faros AI ("Harness Engineering: Making AI Coding Agents Work in 2026"); corroborated independently by GitHub's VS Code team writeup of the Copilot coding harness and multiple 2026 academic surveys (Meng et al., 22-harness-system survey).

DOCUMENTED

Concretely, what the harness does that the model can't

When a model's output says "edit this file," it is the harness that computes the diff and writes it to disk. When the model says "run this command," it is the harness that spawns the process, captures stdout/stderr, and feeds the result back in. The harness decides how much conversation history to keep versus compact away, which tools are exposed to the model at all, and whether a given action needs a human's sign-off before it executes. None of this is model behavior — it's software engineering around the model, and it is where most of 2026's production agent engineering investment is actually going.

Why harness defects, not model limits, cause most failures

Industry analysis circulating in 2026 attributes roughly two-thirds of enterprise agent failures to harness-level defects — specifically *context drift* (the model loses track of what's actually true as conversation grows), *schema misalignment* (tool outputs don't match what the model expects), and *state degradation* (long-running tasks losing coherent state). The practical implication: throwing a more capable model at a broken harness yields diminishing returns, because the harness — not raw reasoning — is usually the bottleneck. **ANALYSIS** — this figure comes from practitioner/industry commentary rather than a single controlled study; treat it directionally.

Enterprise governance: the part that matters for platform teams

As harnesses move from personal coding assistants into pipeline-native, production-facing agents, three concerns dominate the 2026 discussion:

- **Excessive agency** — OWASP's LLM06:2025 category covers over-provisioned tool access, unnecessary permissions, and missing approval mechanisms; it has become the standard checklist for auditing a harness's permission scope against least-privilege.
- **Agent identity** — leading platforms are converging on giving each agent its own scoped credential set (not inheriting the invoking user's full permissions), so an agent's blast radius is bounded regardless of what its prompt says.
- **Approval fatigue** — Anthropic has published on this specifically: when users rubber-stamp roughly 93% of permission prompts, per-action approval stops being a meaningful safety control. The emerging alternative is a two-stage classifier — a fast, cheap gate on every action, with expensive chain-of-thought reasoning reserved only for actions that gate flags as risky.

MCP as the connective tissue

The Model Context Protocol has become the de facto standard for how a harness exposes tools and context to a model, and — increasingly — how one platform's harness talks to another's. By mid-2026, MCP servers exist for essentially every major DevOps surface: GitHub, Azure DevOps, CircleCI, Backstage (via a dedicated MCP Actions Backend plugin), Google Cloud services, and Harness itself. This is what lets an agent running in Cursor, Claude Code, or Copilot reach into a CI/CD platform's live pipeline data without that platform needing to build a bespoke integration for every possible AI client.

DOCUMENTED

Part 2 — How the DevOps/Platform Vendor Landscape Is Adopting It

All ten companies from the original comparison set below now claim some form of AI-agent capability. What differs sharply is **how deep the harness goes** — whether it's a chat assistant bolted onto the product, a governed execution layer where agents run as first-class, audited citizens, or (for several of the older CNCF projects) essentially nothing native at all.

1. GitHub Actions / GitHub Copilot

The deepest, most publicly benchmarked harness in the category.

GitHub's "agentic harness" is a single shared component powering Copilot CLI, the Copilot app, and code review — improve it once, every surface benefits. Custom agents are defined as Markdown files with YAML frontmatter (`.github/agents/*.agent.md`), scoped at repo or org level, and can be picked up by Copilot's cloud agent, VS Code, JetBrains, Xcode, and other IDEs. Enterprise governance runs through an "agent control plane" for visibility/audit, MCP server registry curation, ephemeral runner enforcement, and firewall allowlisting for cloud agents. GitHub publishes controlled cross-model benchmarks (Claude, GPT) of its own harness's efficiency — unusually transparent for the category. **DOCUMENTED**

2. GitLab

Broadest lifecycle-wide orchestration model, GA'd January 2026.

GitLab Duo Agent Platform reached general availability with a distinct architecture: **Agents** (single-purpose, e.g. Planner Agent, Security Analyst Agent) versus **Flows** (multi-agent chains automating a whole task, like turning an open issue into a merge request end to end). All agents query **GitLab Orbit**, a lifecycle context graph connecting code, pipelines, and deployments — architecturally the closest direct analog to Harness's Software Delivery Knowledge Graph. Notably supports external agents (Claude Code, Codex CLI) natively inside the GitLab workflow rather than only first-party agents, and an AI Catalog for publishing/versioning custom agents org-wide. **DOCUMENTED**

3. Azure DevOps

Explicitly the "legacy" half of a two-product AI strategy — Microsoft is steering agent investment toward GitHub.

Microsoft's own 2026 messaging is unusually direct: "AI is the forcing function" for migrating repos to GitHub, where "Copilot, agents, and new AI workflows ship first." Azure DevOps gets an MCP server (preview) exposing work items/builds/PRs to agentic workflows, GitHub Copilot custom-agent selection inside Azure Boards, and an Azure SRE Agent for reliability operations — but Microsoft is actively running an Enterprise Live Migration tool to move teams' source control to GitHub specifically to "unlock the full agentic development experience." Treat Azure DevOps as receiving maintenance-level AI investment, not first-line harness development. **DOCUMENTED**

4. CircleCI

Positions itself specifically as validation infrastructure for agent-generated code, not as an agent-builder.

CircleCI's AI bet is narrower and arguably more defensible than "add a chatbot": the **Chunk** agent autonomously analyzes pipelines for flaky tests, failed builds, and configuration drift, proposing fixes validated in the customer's own environment before being applied. **Chunk Sidecars** (mid-2026) push that validation earlier — into the coding agent's own inner loop, running tests/linting/formatting before code is even committed. A CircleCI MCP server (also available via AWS Bedrock AgentCore) lets external agents in Cursor, Claude Desktop, or Windsurf trigger builds and diagnose failures in natural language. The framing is consistent: as AI-generated code accelerates commit velocity, CircleCI sells itself as the checkpoint that keeps validation from becoming the bottleneck. **DOCUMENTED**

5. Jenkins

Minimal native AI; whatever agent capability exists comes from the plugin ecosystem, not the core project.

No evidence of a first-party Jenkins agent harness comparable to the above emerged in current research. Jenkins' AI story in 2026 is largely third-party: LLM-assisted tools exist for converting Jenkinsfiles to other platforms' pipeline formats (notably GitLab's own "Convert Jenkins to GitLab CI/CD" flow, built to migrate people *off* Jenkins), and generic AI coding assistants can of course write Groovy pipeline scripts like any other code. For organizations with heavy Jenkins investment, the practical AI story is "bring your own IDE agent to write Jenkinsfiles," not a governed, pipeline-native agent execution layer. **ANALYSIS**

6. ArgoCD

No native agent harness; GitOps' pull-based reconciliation model is itself somewhat agent-adjacent by design.

ArgoCD remains a CNCF-graduated, Kubernetes-native GitOps reconciler with no evidence of a first-party AI agent layer. Worth noting architecturally: ArgoCD's pull-based "Git is the source of truth, cluster reconciles to match it" model is philosophically close to how a well-designed agent harness should treat state — declarative, diffable, convergent — even though ArgoCD itself isn't AI-driven. Where agentic workflows touch ArgoCD today, it's typically an external agent (via a CI tool's MCP server, or a platform like Harness/GitLab) generating or modifying the Kubernetes manifests that ArgoCD then reconciles, not an agent embedded in ArgoCD itself. **ANALYSIS**

7. Backstage

The most-discussed "is this architecture agent-ready?" case in the category — and the honest answer as of 2026 is "not yet, and not without rework."

Backstage shipped real primitives toward agent-readiness: v1.40's **Actions Registry** lets plugins register typed, schema-validated actions with permission enforcement, and `@backstage/plugin-mcp-actions-backend` exposes those actions as MCP tools so agents in Cursor or Claude can discover and invoke them. v1.43 added OAuth-based Dynamic Client Registration, replacing static-token auth with real per-user identity for MCP calls. However, Backstage's own documentation and independent platform-engineering commentary describe the MCP integration as "highly experimental," and a widely-circulated 2026 industry critique argues the deeper problem is architectural: Backstage centers the UI with everything else serving it, whereas an agent-native platform needs the catalog/context-graph to be the center, with the UI as just one consumer among agent-native interfaces. That's characterized as a re-architecture, not a version bump. **DOCUMENTED** for the shipped primitives; **ANALYSIS** for the architectural critique, which is one influential commentator's framing, not a Backstage project position.

8. Humanitec

Governance-first platform orchestrator; positions itself as what runs underneath a portal (including Backstage) rather than as an agent-harness product itself.

Humanitec is a Platform Orchestrator focused on standardizing infrastructure operations and self-service, commonly paired with Backstage via official plugins (workload/environment/resource visibility inside a Backstage portal) rather than replacing it. No evidence surfaced of a first-party LLM agent harness comparable to GitLab's or GitHub's; Humanitec's relevance to the agent-harness conversation is more indirect — it's frequently cited as the kind of deterministic, policy-driven "execution substrate" that agent-facing platforms (per the Backstage critique above) are being pushed toward building. **ANALYSIS**

9. Spinnaker

Mature multi-cloud CD platform; effectively no 2026 AI-agent narrative in current research.

Originally built at Netflix for multi-cloud continuous delivery with sophisticated deployment strategies (canary, blue/green, highlander), Spinnaker shows no evidence of native agent-harness investment as of mid-2026. Multiple comparison sources instead frame Spinnaker itself as the thing organizations are moving away from toward lighter, more AI-native alternatives, citing setup complexity and maintenance burden as the more pressing 2026 conversation for Spinnaker users rather than AI capability. **ANALYSIS**

10. Tekton

Kubernetes-native pipeline building blocks; AI activity is happening at the layer above it, not inside it.

Tekton provides the underlying custom-resource pipeline engine that other projects (Jenkins X, OpenShift Pipelines) build on top of; it has no native agent harness of its own. The one concrete AI use case found in current research is LLM-assisted *migration into* Tekton — generative tooling trained on paired pipeline examples to convert other platforms' pipeline definitions into Tekton YAML. Separately, emerging CNCF-ecosystem projects like OpenChoreo (an Internal Developer Platform in the CNCF Sandbox as of early 2026) are exposing MCP servers on top of Kubernetes-native stacks for agent-driven deployment and reasoning — a sign that agent-harness capability in the Kubernetes-native world is being built as a layer above tools like Tekton, not inside them. **DOCUMENTED** for what exists; **ANALYSIS** for the layering interpretation.

Part 3 — The Pattern Across All Ten

Tier	Vendors	What characterizes it
Deep, governed agent harness	GitHub, GitLab, Harness (from Part 1's research), CircleCI	First-party agents with scoped identity/credentials, audit trails, MCP exposure, and — critically — the same approval/RBAC gates that govern human-triggered actions also gate the agent
Agent-curious, architecturally unresolved	Backstage, Azure DevOps	Real primitives shipped (MCP actions, custom agent selection) but either explicitly "experimental" (Backstage) or explicitly secondary to a sibling product getting the real investment (Azure DevOps vs. GitHub)
No native harness; agents operate above/around it	Jenkins, ArgoCD, Spinnaker, Tekton, Humanitec	Mature, often CNCF-grade infrastructure that agentic tooling treats as a target to generate config for or reconcile against, rather than a platform that runs agents itself

The throughline back to Part 1: the vendors in the top tier all converged, independently, on the same governance pattern that Part 1 identifies as the hard part of harness engineering — scoped per-agent identity,

policy gates shared with human workflows, and audit trails — rather than on model choice or raw agent capability. That convergence is itself evidence for Part 1's central claim: the harness, not the model, is where 2026's real engineering investment is going.

Company profiles are necessarily compressed for comparability; treat "no evidence found" for Jenkins/ArgoCD/Spinnaker/Tekton as a statement about current public research, not a certainty that no such capability exists anywhere in those ecosystems' plugin/vendor long tail.